ĐẠI HỌC QUỐC GIA TP HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Giới Thiệu Đề Tài

Ứng dụng xử lý ảnh trong giải mã mã vạch EAN/UPC với giao diện đồ họa MATLAB

Ngày 16 tháng 1 năm 2025

Giới thiệu thành viên

- Lê Thị Thuỳ Nhung MSSV: 22110151
- Nguyễn Hà Thu Quỳnh MSSV: 22110185
- Nguyễn Thuỳ Uyên Nhi MSSV: 22110148
- Ngô Thuý Quỳnh MSSV: 22110184

Lời nói đầu

Trong thời đại số hóa ngày nay, mã vạch đã trở thành một phần không thể thiếu trong việc quản lý thông tin sản phẩm và hàng hoá. Các chuẩn mã vạch EAN/UPC là những chuẩn phổ biến nhất được sử dụng rộng rãi trong lĩnh vực bán lẻ và logistics. Với sự phát triển mạnh mẽ của công nghệ xử lý ảnh, việc tự động hóa quá trình nhận diện và giải mã mã vạch đã đạt được những tiến bộ đáng kể.

Dự án này tập trung vào việc nghiên cứu và phát triển một ứng dụng xử lý ảnh có khả năng giải mã mã vạch EAN/UPC, được xây dựng trên nền tảng MATLAB với giao diện đồ họa người dùng (GUI) thân thiện. Thông qua dự án này, chúng tôi mong muốn mang lại một giải pháp đơn giản nhưng hiệu quả cho việc nhận diện mã vạch trong các hình ảnh chụp, giúp cải thiện độ chính xác và tốc độ trong quá trình quản lý thông tin sản phẩm.

Tổng quan về đề tài

1.1 Khái niệm mã vạch

- Mã vạch là một đại diện trực quan của dữ liệu, bao gồm một chuỗi các đường thẳng song song (các thanh) và các khoảng trắng có độ rộng thay đổi, cùng với các con số hoặc ký tự được in hoặc hiển thị bên dưới. Mã vạch được sử dụng rộng rãi trong nhiều ngành công nghiệp để nhận diện, kiểm soát hàng tồn kho và thu thập dữ liệu.
- Trong thương mại hiện đại, mã vạch cung cấp một cách nhanh chóng để nhận diện sản phẩm, giúp thanh toán nhanh hơn, quản lý hàng tồn kho hiệu quả và tối ưu hoá logistics.

1.2 Các loại mã vạch

- **EAN-13**: Thường được sử dụng trong ngành bán lẻ.

EAN BARCODE

BARCODE WORLD

- **UPC-A**: Sử dụng rộng rãi tại Mỹ cho các sản phẩm bán lẻ.

- **Code 39**: Thường được sử dụng trong ngành công nghiệp ô tô và quốc phòng.

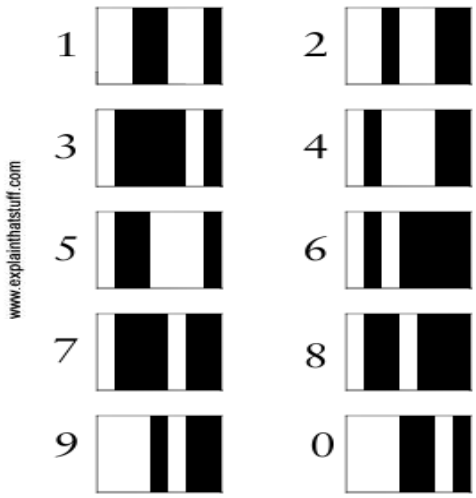
- **Code 128**: Được sử dụng trong logistics và vận chuyển.

1.3 Cấu tạo của mã vạch

Một hệ thống mã vạch như thế này có ba phần chính:

1. **Hệ thống cơ sở dữ liệu:**
    - Máy tính trung tâm chạy hệ thống lưu trữ, giữ dữ liệu của tất cả sản phẩm bao gồm thông tin về người sản xuất, giá cả và số lượng tồn kho.
2. **Mã vạch trên sản phẩm:**
    - Các mã vạch được in trên tất cả sản phẩm.
3. **Máy quét mã vạch:**
    - Một hoặc nhiều máy quét tại quầy thanh toán để đọc các mã vạch.

Khi nhìn vào mã vạch, ta không thể phân biệt rõ đâu là phần bắt đầu và kết thúc. Tuy nhiên, thực tế mỗi chữ số trong số sản phẩm được mã hóa chính xác với khoảng cách 7 đơn vị, bao gồm các thanh và khoảng trống xen kẽ nhau một cách hệ thống.



Ứng dụng của mã vạch

Quản lý hàng tồn kho

Mã vạch giúp tự động hóa quá trình kiểm kê và quản lý kho, giúp theo dõi số lượng sản phẩm chính xác và nhanh chóng. Nhân viên có thể quét mã vạch của các sản phẩm để cập nhật tình trạng tồn kho mà không cần nhập liệu thủ công.

Hệ thống bán hàng (POS)

Mã vạch được sử dụng trong các hệ thống bán hàng để nhanh chóng tra cứu thông tin sản phẩm, tính toán giá cả và in hóa đơn. Điều này giúp tăng tốc độ thanh toán và giảm thiểu sai sót trong giao dịch.

Theo dõi và quản lý vận chuyển

Mã vạch giúp theo dõi lộ trình của các sản phẩm hoặc gói hàng trong quá trình vận chuyển. Điều này giúp tối ưu hóa các quy trình logistics và giảm thiểu thất lạc hoặc chậm trễ.

Ngành y tế

Trong y tế, mã vạch được sử dụng để theo dõi bệnh nhân, thuốc men và thiết bị y tế. Điều này giúp tăng cường độ chính xác trong việc cung cấp thuốc và kiểm tra bệnh án.

Quản lý thư viện

Mã vạch giúp quản lý các đầu sách trong thư viện, cho phép việc mượn và trả sách trở nên nhanh chóng và hiệu quả hơn. Các mã vạch trên sách giúp nhân viên thư viện dễ dàng kiểm tra thông tin và tình trạng sách.

Chứng nhận sản phẩm và chống hàng giả

Mã vạch có thể được sử dụng để xác minh tính hợp pháp và nguồn gốc của sản phẩm, đặc biệt trong các ngành như thực phẩm, dược phẩm và hàng hoá cao cấp. Điều này giúp giảm thiểu hàng giả và hàng nhái trên thị trường.

Giới thiệu về ứng dụng di động

Các ứng dụng quét mã vạch trên điện thoại di động có thể giúp người tiêu dùng tra cứu thông tin sản phẩm, so sánh giá cả và tìm kiếm các khuyến mãi. Điều này mang lại sự tiện lợi cho người tiêu dùng khi mua sắm.

Tự động hóa trong sản xuất

Mã vạch được sử dụng trong các dây chuyền sản xuất để theo dõi quá trình sản xuất và đảm bảo sản phẩm được hoàn thành đúng tiến độ, đúng chất lượng.

Quản lý tài sản

Mã vạch có thể giúp theo dõi và quản lý tài sản trong các doanh nghiệp, giúp dễ dàng nhận diện, kiểm tra và bảo trì tài sản trong suốt vòng đời của chúng.

Nhờ vào khả năng quét và giải mã nhanh chóng, mã vạch đã trở thành một công cụ không thể thiếu trong các ngành công nghiệp hiện đại, giúp tiết kiệm thời gian, giảm thiểu sai sót và nâng cao hiệu quả công việc

Lý do chọn đề tài

1. **Ứng dụng rộng rãi trong thực tiễn**: Mã vạch được sử dụng trong nhiều lĩnh vực như bán lẻ, quản lý kho, y tế, logistics và vận chuyển.
2. **Khả năng tích hợp với hệ thống quản lý**: Mã vạch có thể tích hợp với các hệ thống quản lý hàng hoá, kiểm kê sản phẩm và thanh toán.
3. **Phát triển kỹ năng lập trình**: Học cách sử dụng các thư viện Python như pyzbar, barcode hoặc OpenCV.
4. **Tính thực tiễn cao**: Dễ thực hiện và có thể phát triển thành các ứng dụng thực tế.
5. **Nâng cao khả năng giải quyết vấn đề**: Rèn luyện khả năng phân tích và tối ưu hóa.
6. **Tính sáng tạo và mở rộng**: Có thể mở rộng thành các ứng dụng như cơ sở dữ liệu, ứng dụng di động hoặc web.

Tóm lại: Đề tài "Ứng dụng mã vạch" không chỉ giúp sinh viên hiểu rõ về một công nghệ phổ biến mà còn mở ra nhiều cơ hội học hỏi và phát triển kỹ năng lập trình, đồng thời giải quyết các vấn đề thực tế trong quản lý sản phẩm và thông tin.

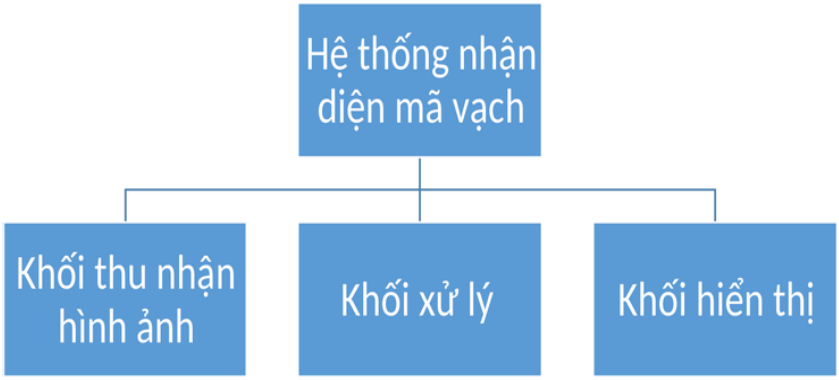
Phạm vi nghiên cứu

- Tập trung vào mã vạch EAN/UPC với các thuật toán xử lý ảnh cơ bản và nâng cao.
- Giao diện đồ họa dễ sử dụng trên MATLAB.
- Tài liệu hướng dẫn sử dụng chi tiết.
- Hiệu quả xử lý mã vạch nhanh chóng và chính xác.

Sơ đồ khối

Một hệ thống nhận diện mã vạch bao gồm ba thành phần chính:

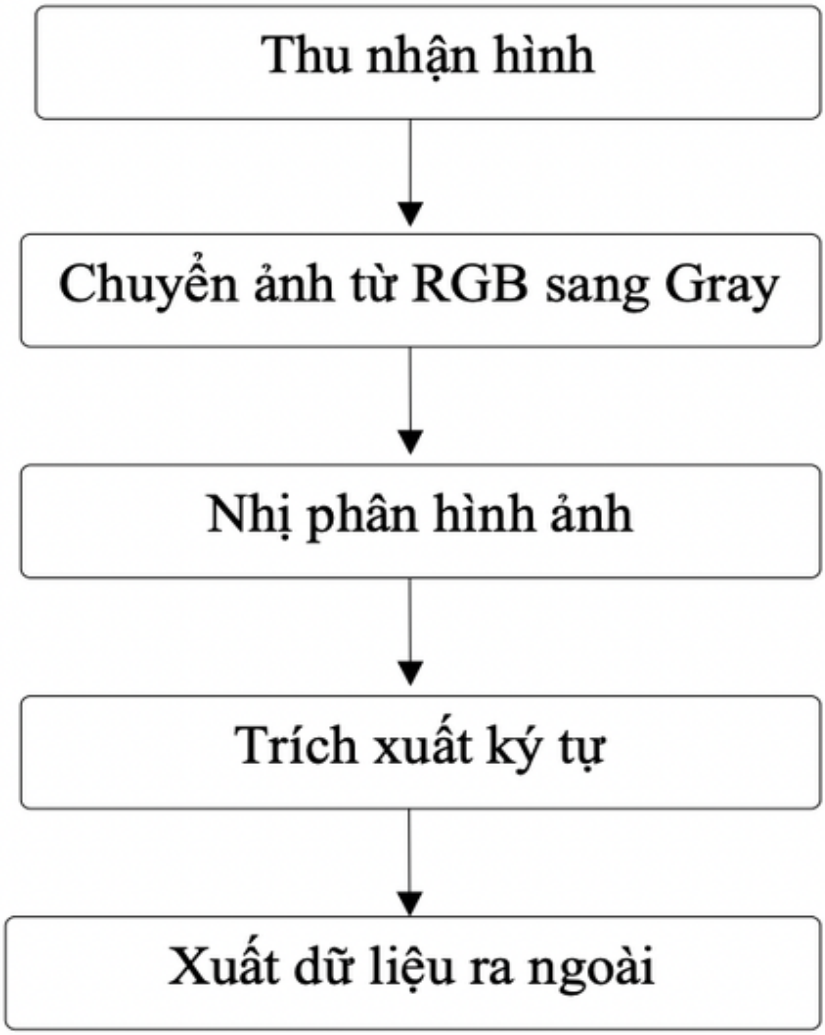
1. **Khối thu nhận hình ảnh:**
   - Bao gồm các thiết bị thu nhận hình ảnh như camera, máy quét, scanner, sensor, hoặc nguồn dữ liệu được lưu trữ trong máy tính.
2. **Khối xử lý:**
   - Dựa vào đặc điểm của mã vạch và quy tắc cấu tạo mã vạch để đưa ra các câu lệnh xử lý hình ảnh thu được từ khối thu.
3. **Khối hiển thị:**
   - Hiển thị dãy mã số đã được xử lý từ hình ảnh thu được ra màn hình cho người dùng.



Sơ đồ giải thuật

Mô hình và cách hoạt động:

1. **Xử lý ảnh tiền xử lý:**
   - Chuyển đổi ảnh từ RGB sang ảnh xám (`rgb2gray`).
   - Chuyển ảnh xám thành ảnh nhị phân (`imbinarize`).
2. **Giải mã mã vạch:**
   - Dựa trên các thuật toán phân tích nhị phân tự phát triển:
     - **BYhorizontal**: Trích xuất dữ liệu theo hướng ngang (horizontal scanning).
     - **Region**: Xác định các vòng thay đổi trong dữ liệu nhị phân.
     - **Collect**: Gom nhóm các vòng và tính toán các đặc trưng.
     - **CheckT**: Xác minh dữ liệu nhị phân dựa trên các quy tắc.
     - **eanupc**: Giải mã mã vạch theo tiêu chuẩn EAN/UPC (thường được sử dụng cho mã vạch hàng hoá).
3. **Không sử dụng học máy:**
   - Không cần mô hình học máy hoặc mạng nơ-ron nhân tạo nào được triển khai. Hoạt động dựa trên các quy tắc và thuật toán mã hoá/giải mã thuần túy.



Quy trình thực hiện

Bước 1: Thu thập dữ liệu

Hình ảnh được lấy ngẫu nhiên từ Internet.

Bước 2: Sử dụng các hàm sau để xử lý ảnh

1. **BYhorizontal.m**

   Cách hoạt động

1. Kiểm tra và chuyển ảnh về dạng nhị phân

- Nếu ảnh chưa phải ảnh nhị phân (islogical(I) == false), thực hiện:
  - Chuyển ảnh màu sang ảnh xám (rgb2gray).
  - Chuyển ảnh xám sang ảnh nhị phân (imbinarize).
- Sau bước này, ảnh chỉ có giá trị 0 (đen) và 1 (trắng).

2. Khởi tạo mảng b để lưu kết quả

- Mảng b có độ dài bằng số cột (NumofColoms), ban đầu chứa các giá trị từ 1:NumofColoms

3. Duyệt qua từng cột của ảnh để lấy dữ liệu nhị phân

- Xác định hàng cần quét:
  - round(NumofRows / horizantalRule) → Lấy một hàng dựa trên horizantalRule.
- Duyệt qua từng pixel của hàng đó (cột x):
  - Nếu pixel màu đen (0), lưu 1 vào b(j).
  - Nếu pixel màu trắng (1), lưu 0 vào b(j).
- Cập nhật chỉ số j để ghi dữ liệu tiếp theo.

4. Trả về kết quả

- Gán b cho c, kết quả là một dãy 0 và 1, biểu diễn vạch mã vạch.

2. Region.m

- **Chức năng của hàm**

- Hàm Region(c) tính toán **số lần giá trị trong vector c thay đổi** giữa các phần tử liên tiếp.
- **Phân tích từng phần**
- **Khởi tạo biến đếm td**

  td = 0;
- td được dùng để đếm số lần có sự thay đổi giữa các phần tử liên tiếp trong c.
- **Duyệt qua các phần tử của c**
  for i = 1:(length(c)-1)
- Vòng lặp chạy từ 1 đến length(c)-1, tức là nó sẽ kiểm tra từng phần tử c(i) với c(i+1).
- **Kiểm tra sự thay đổi giá trị**
  if c(i) ~= c(i+1)
- td = td + 1;
- end
- Nếu giá trị của phần tử c(i) khác với c(i+1), nghĩa là có sự thay đổi, thì tăng biến đếm td lên 1.
- **Trả về kết quả td**

3. Collect.m

Cách hoạt động của hàm

1. **Khởi tạo biến**

- ii = 1: Chỉ số chạy qua các phần tử trong c.
- s = [1:td]: Khởi tạo mảng s có td phần tử (các giá trị sẽ được cập nhật sau).

2. **Duyệt qua từng vùng (vùng liên tiếp của 0 hoặc 1)**

- Vòng lặp for k=1:td chạy td lần, mỗi lần xử lý một vùng liên tiếp.
- j = 0: Biến đếm số lượng phần tử liên tiếp có cùng giá trị.

3. **Kiểm tra giá trị tại c(ii)**

- Nếu **c(ii) == 0**:
  - Duyệt qua c bằng vòng while, đếm số lượng 0 liên tiếp.
  - Khi gặp 1, lưu số lượng 0 đã đếm vào s(k).
- Nếu **c(ii) == 1**:
  - Tương tự, đếm số lượng 1 liên tiếp cho đến khi gặp 0.
  - Lưu kết quả vào s(k).

4. **<u>CheckT.m</u>**

Cách hoạt động

1. Xác định k (Duyệt từ trái sang phải)

- Bắt đầu với k = 1.
- Duyệt qua s từ phần tử **đầu tiên** đến (length(s) - 11), nghĩa là xét các nhóm **12 số liên tiếp**.
- Trong mỗi nhóm:
  - Tìm **số nhỏ nhất** (rec) trong nhóm 12 phần tử.
  - Chia từng số trong nhóm cho rec, làm tròn kết quả và lưu vào mảng d.
  - Kiểm tra khoảng dao động (max(d) - min(d)).
    - Nếu **lớn hơn 4**, tăng k và tiếp tục duyệt.
    - Nếu **nhỏ hơn hoặc bằng 4**, dừng lại (break).

2. Xác định g (Duyệt từ phải sang trái)

- Bắt đầu với g = 1.
- Duyệt qua s từ **phần tử cuối cùng** về phần tử thứ 12 (ngược với k).
- Trong mỗi nhóm **12 số liên tiếp**:
  - Tìm số nhỏ nhất (rec).
  - Chia từng số trong nhóm cho rec, làm tròn kết quả và lưu vào d.
  - Kiểm tra sự thay đổi (max(d) - min(d)).
    - Nếu **lớn hơn 4**, tăng g và tiếp tục duyệt.
    - Nếu **nhỏ hơn hoặc bằng 4**, dừng lại.

Ý nghĩa của k và g

- **k xác định từ đầu chuỗi s vị trí đầu tiên có sự ổn định**.
- **g xác định từ cuối chuỗi s vị trí đầu tiên có sự ổn định**.

Hai giá trị này có thể dùng để **xác định phần ổn định của mã vạch**.

5. **<u>eanupc.m</u>**

Chức năng chính

Hàm eanupc(p, k) có nhiệm vụ trích xuất 12 chữ số từ một mã vạch **EAN-13 hoặc UPC-A** dựa trên một dãy số đầu vào p (đại diện cho các thanh mã vạch) và một vị trí bắt đầu k.

Cách hoạt động

1. **Khởi tạo hai mảng doc1 và doc2**:
   - doc1 chứa 6 số đầu tiên của mã vạch.
   - doc2 chứa 6 số còn lại.
2. **Duyệt qua p và trích xuất 6 số đầu (doc1)**:
   - Vòng lặp đầu tiên chạy từ k+3 đến k+23 với bước nhảy 4, tức là đọc các nhóm 4 số một lần.
   - Mỗi nhóm 4 số này được so sánh với các mẫu để xác định con số tương ứng từ 0 đến 9.
   - Kết quả lưu vào doc1.
3. **Duyệt qua p và trích xuất 6 số cuối (doc2)**:
   - Vòng lặp thứ hai chạy từ k+32 đến k+52, cũng đọc từng nhóm 4 số một lần.
   - Quá trình giải mã tương tự như với doc1.
   - Kết quả lưu vào doc2.

4. **Ghép kết quả của doc1 và doc2 thành result**:

   result = [doc1 doc2] chứa 12 số của mã vạch.

6. **<u>giao_dien.m</u>**

1. Khởi tạo giao diện (giaodien.m)

Khi chạy chương trình, MATLAB tạo một GUI với các nút bấm để điều khiển.

Các bước khởi tạo quan trọng:

1. Xác định trạng thái của GUI (gui_State):
   - gui_Singleton = 1 → Chỉ cho phép chạy một cửa sổ GUI duy nhất.
   - mfilename → Lấy tên của file đang chạy (giaodien.m).
   - gui_OpeningFcn → Hàm khởi động GUI (giaodien_OpeningFcn).
   - gui_OutputFcn → Hàm lấy kết quả (giaodien_OutputFcn).
2. Gọi gui_mainfcn(gui_State, varargin{:}):
   - Nếu có đối số đầu vào (varargin), GUI sẽ xử lý theo đó.

2. Mở giao diện (giaodien_OpeningFcn)

Chương trình chạy vào giaodien_OpeningFcn khi giao diện khởi động.

Các bước chính:

1. Gán handles.output = hObject → Lưu trạng thái của GUI.
2. Lưu trạng thái GUI (guidata(hObject, handles)).

Mục đích: Chuẩn bị GUI, sẵn sàng nhận lệnh từ người dùng.

3. Nhận kết quả từ GUI (giaodien_OutputFcn)

Khi cần lấy kết quả từ GUI, MATLAB gọi giaodien_OutputFcn.

- Nó trả về handles.output, là biến lưu trạng thái GUI.

Mục đích: Cho phép chương trình bên ngoài lấy kết quả từ giao diện.

4. Khởi động camera (start_cam_Callback)

Khi nhấn nút "Start Camera", chương trình kết nối với camera hoặc thiết bị quay video.

Các bước xử lý camera:

1. Xóa bộ nhớ thiết bị (imaqreset) để tránh lỗi khi kết nối lại.
2. Gọi imaqhwinfo('winvideo',2) để lấy thông tin thiết bị video.
3. Mở camera (videoinput('winvideo', 2, 'YUY2_640x480')):
   - winvideo: Chọn driver camera trên Windows.
   - 2: Chỉ định thiết bị camera (có thể thay đổi).
   - 'YUY2_640x480': Định dạng video đầu vào.
4. Thiết lập chế độ màu (vid.ReturnedColorSpace = 'rgb').

5. Hiển thị hình ảnh từ camera (preview(vid, hImage)).

Mục đích: Bật camera, hiển thị video trên GUI.

5. Chụp ảnh (capture_Callback)

Khi nhấn "Capture", chương trình chụp ảnh từ camera và hiển thị trên giao diện.

Các bước xử lý:

1. Lấy ảnh (S = getsnapshot(vid)) từ camera.
2. Hiển thị ảnh (imshow(S)) trên giao diện.
3. Lưu ảnh vào file (imwrite(S, 'imagename2.jpg')).
4. Lưu ảnh vào handles.S để sử dụng trong xử lý mã vạch.

Mục đích: Chụp ảnh từ camera để xử lý mã vạch.

6. Tải ảnh (loadimage_Callback)

Cho phép người dùng chọn ảnh từ máy tính.

Các bước xử lý:

1. Hiển thị hộp thoại chọn file (uigetfile).
2. Đọc ảnh (imread) và hiển thị (imshow) trên giao diện.
3. Lưu ảnh vào handles.S để xử lý mã vạch.

Mục đích: Tải ảnh có sẵn để nhận diện mã vạch.

7. Nhận diện mã vạch (result_Callback)

Xử lý ảnh và trích xuất mã vạch.

Các bước xử lý:

1. Lấy ảnh (handles.S) và xác định kích thước (size).
2. Tiền xử lý ảnh:
   - Cắt ảnh (imcrop) lấy vùng mã vạch.
   - Chuyển ảnh xám (rgb2gray) và tăng tương phản (imadjust).
   - Nhị phân hóa (im2bw) để chỉ còn đen - trắng.
3. Xử lý mã vạch theo từng dòng quét:
   - BYhorizontal: Trích xuất dữ liệu nhị phân từ mã vạch.
   - Region: Xác định các vùng mã vạch.
   - Collect: Đo độ rộng vạch đen/trắng.
   - CheckT: Kiểm tra tính nhất quán.
4. Giải mã mã vạch (eanupc):
   - Chia tỷ lệ độ rộng vạch (rec = s(k)) để chuẩn hóa.
   - Nếu dữ liệu hợp lệ ((td - g - k) == 57), gọi eanupc(p, k).
5. Hiển thị mã vạch trên GUI.

Mục đích: Xử lý và hiển thị kết quả nhận diện mã vạch.

8. Thoát chương trình (exitprogram_Callback)

Xóa GUI khi người dùng nhấn "Exit".

Mục đích: Đóng giao diện và kết thúc chương trình.

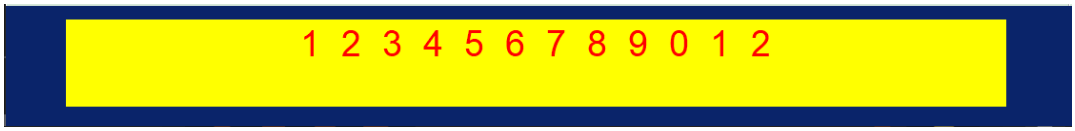
7. **giao_dien.fig**
   - **Chức năng:** Định dạng giao diện đồ họa.
   - File .fig này chứa thiết kế giao diện (GUI) của hệ thống, được sử dụng bởi giao_dien.m để hiển thị.

Bước 3: Phân tích và giải mã mã vạch

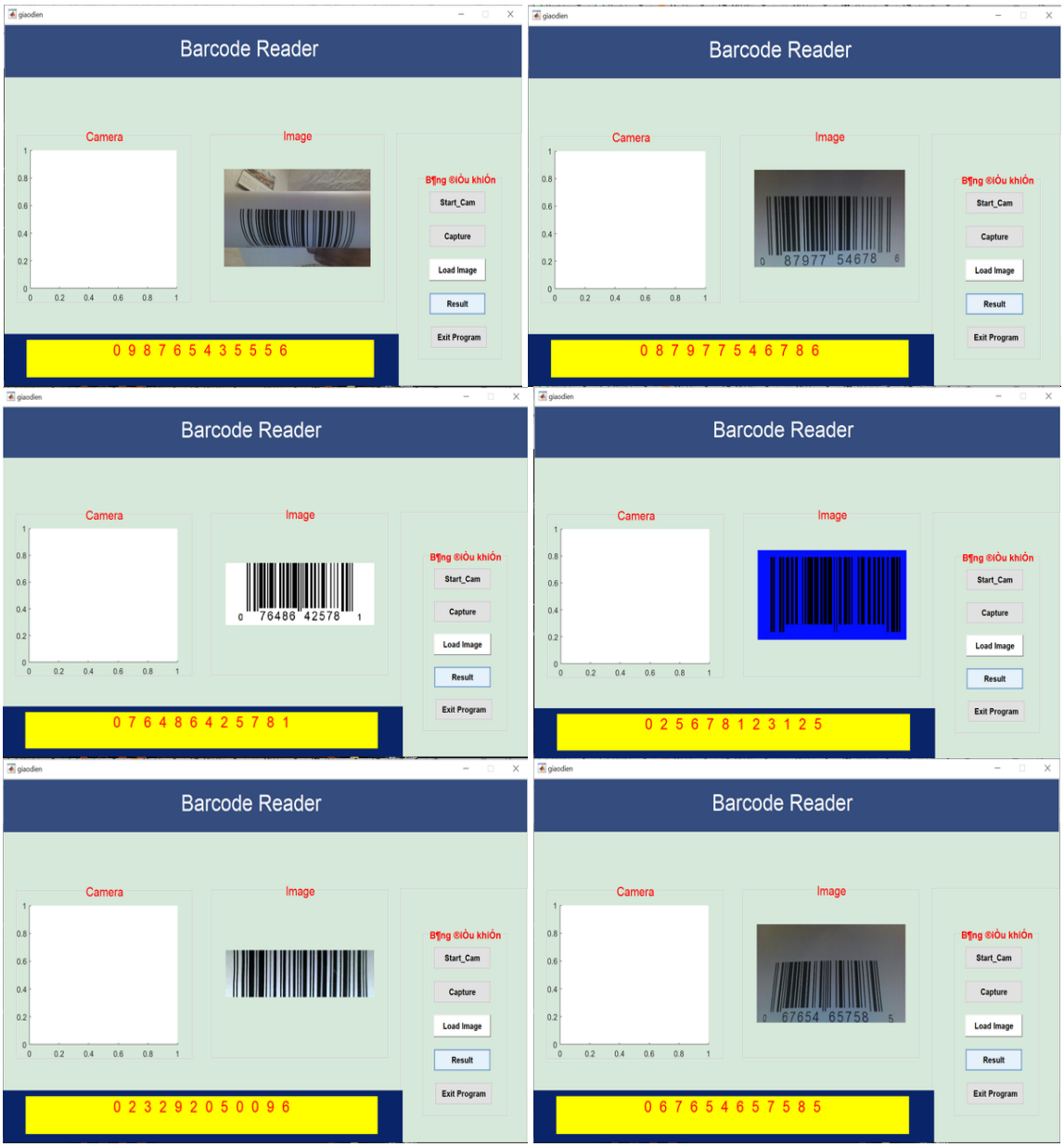
1. Giao diện GUI ban đầu

2. Hình ảnh được đưa vào từ database

3. Kết quả xuất ra màn hình



Một số kết quả xuất ra màn hình



Đánh giá mô hình:

1. **Đơn giản:**
   - Dễ thực hiện và không cần thư viện hoặc mô hình học máy phức tạp, chỉ cần MATLAB cơ bản.
2. **Hiệu quả:**
   - Với dữ liệu có độ tương phản tốt, chương trình có thể giải mã nhanh chóng.
3. **Khả năng triển khai:**
   - Không cần tập huấn, chỉ cần cài đặt MATLAB và chạy mã.
4. **Nhược điểm:**
   - Hạn chế trong khả năng nhận diện mã vạch khi chất lượng ảnh kém hoặc mã vạch bị mờ/nhòe.
   - Khó xử lý các loại mã phức tạp như QR code.

Đề xuất cải thiện:

- Sử dụng các mô hình học máy hoặc AI hiện đại như:
  - **MATLAB:** Hỗ trợ thư viện Computer Vision dễ dàng xử lý mã vạch và QR code.
  - **Mô hình học sâu (Deep Learning):** YOLO (You Only Look Once) cho nhận diện mã vạch, hoặc CNN (Động Lắp Tích Chồng).
  - Triển khai nhận ảnh bằng webcam

Kết luận

Trong bài báo cáo này, chúng tôi đã nghiên cứu và triển khai ứng dụng xử lý ảnh nhằm giải mã mã vạch EAN/UPC với giao diện đồ họa trên MATLAB. Quá trình thực hiện bao gồm các bước từ xử lý ảnh, trích xuất dữ liệu mã vạch đến giải mã và hiển thị kết quả. Dựa trên các thử nghiệm và đánh giá, có thể rút ra một số kết luận chính như sau:

Hiệu quả của hệ thống

- Ứng dụng đã chứng minh khả năng giải mã chính xác các mã vạch EAN/UPC phổ biến. Các thuật toán xử lý ảnh cơ bản và nâng cao được tích hợp để đảm bảo nhận diện ngay cả khi chất lượng ảnh thấp hoặc mã vạch bị lệch.

Tính thực tiễn

- Giao diện đồ họa MATLAB thân thiện, dễ sử dụng, giúp người dùng không cần hiểu biết sâu về kỹ thuật vẫn có thể vận hành hệ thống. Hệ thống có tiềm năng ứng dụng trong nhiều lĩnh vực như bán lẻ, quản lý kho, và logistics.

Hạn chế và cải tiến

- Một số hạn chế vẫn tồn tại như xử lý mã vạch bị nhiễu hoặc bị mờ. Trong tương lai, hệ thống có thể được cải thiện bằng cách tích hợp các thuật toán xử lý ảnh hiện đại như mạng nơ-ron tích chập (CNN) hoặc học sâu (Deep Learning) để nâng cao độ chính xác.

Tầm quan trọng của công nghệ

- Việc ứng dụng xử lý ảnh vào giải mã mã vạch không chỉ góp phần tự động hóa các quy trình mà còn giúp tiết kiệm thời gian và giảm thiểu sai sót trong vận hành.

Tài liệu tham khảo

1. Studocu
2. Make Use Of
3. Geeks for Geeks